

Dynamic Programming - 1

- Karun Karthik

Contents

0. Introduction
- ~~1. Climbing Stairs~~
2. Fibonacci Number
3. Min Cost Climbing Stairs
4. House Robber
5. House Robber - II
6. Nth Tribonacci Number
7. 0-1 Knapsack
8. Partition Equal Subset Sum
9. Target Sum
10. Count no of Subsets with given Difference
11. Delete and Earn
12. Knapsack with Duplicate Items
13. Coin Change - II
14. Coin Change
15. Rod cutting

Introduction

Dynamic programming is a technique to solve problems by breaking it down into a collection of sub-problems, solving each of those sub-problems just once and storing these solutions inside the cache memory in case the same problem occurs the next time.

Dynamic Programming is mainly an optimization over plain recursion .

Wherever we see a recursive solution that has repeated calls for same inputs, we can optimize it using Dynamic Programming.

This simple optimization reduces the time complexities from exponential to polynomial.

There are two different ways to store our values so that they can be reused at a later instance. They are as follows:

1. Memoization or the Top Down Approach.
2. Tabulation or the Bottom Up approach.

In Memoization we start from the extreme state and compute result by using values that can reach the destination state i.e the base state.

In Tabulation we start from the base state and then compute results all the way till the extreme state.

Note: To store the intermediate results we can use Array, Matrix, Hashmap etc., all we need is data storage and retrieval with a specific key.

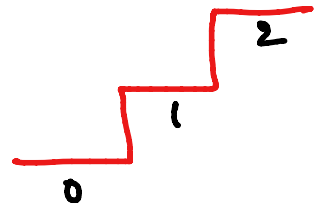
How to find the use case of Dynamic Programming?

You can use DP if the problem can be,

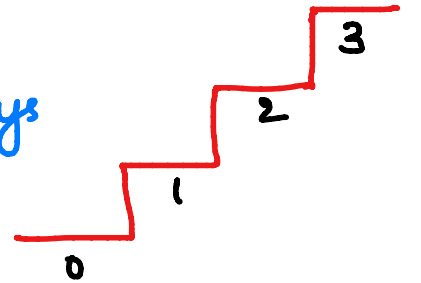
1. Divided into sub-problems
2. Solved using a recursive solution
3. Containing repetitive sub-problems

① Climbing Stairs → Given a value 'N', find the number of ways to reach N & jumps possible are ONE or TWO.

eg $n=2 \Rightarrow$
 $0 \xrightarrow{1} 1 \xrightarrow{1} 2$
 $0 \xrightarrow{2} 2$ } for $N=2$
 we have 2 ways

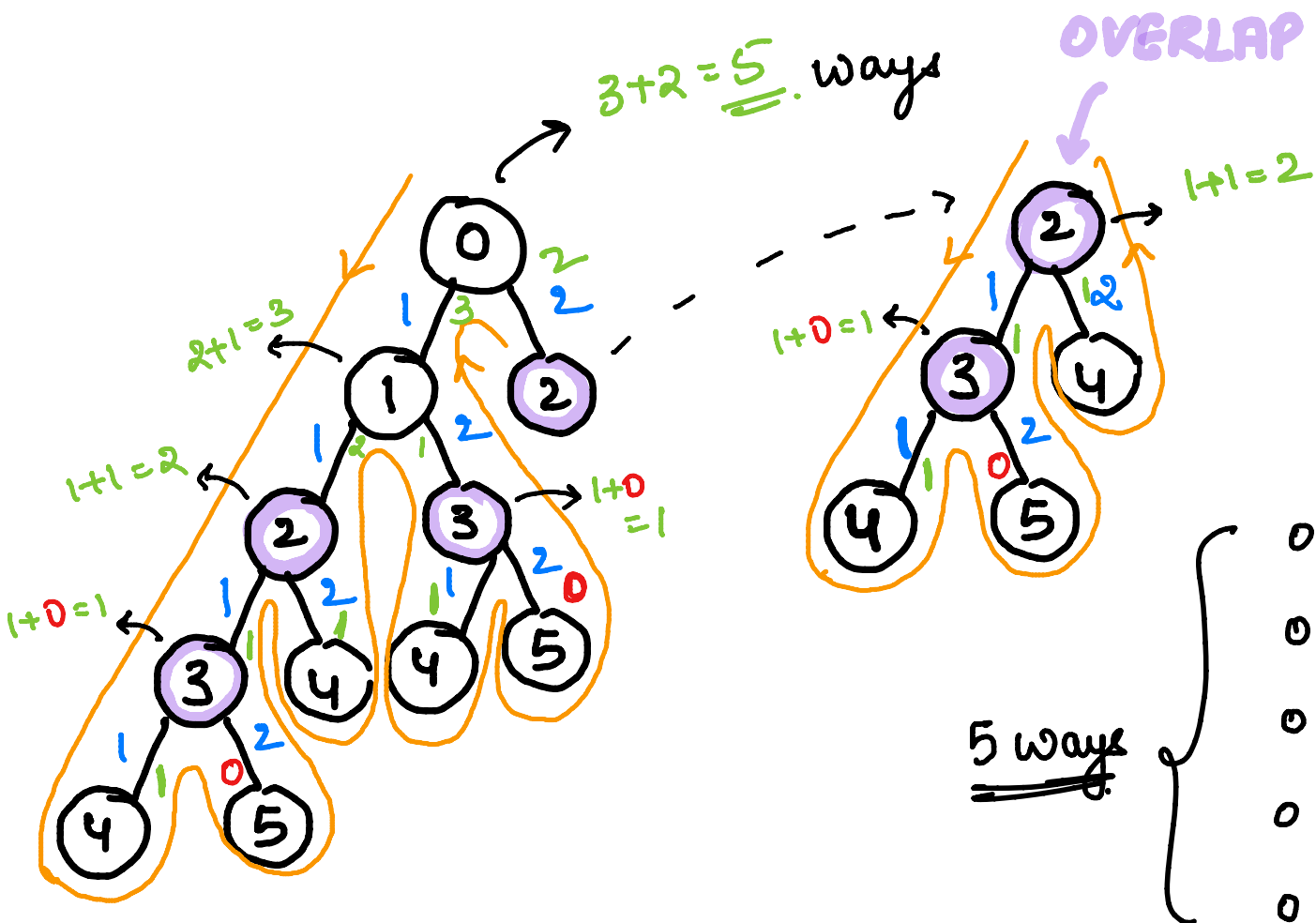
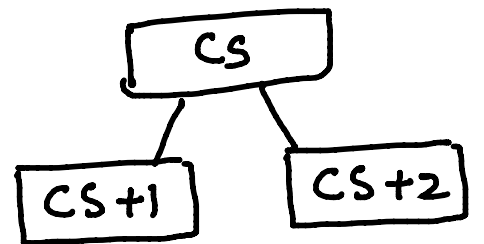
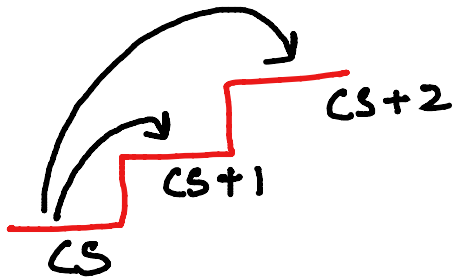


$n=3 \Rightarrow$
 $0 \xrightarrow{1} 1 \xrightarrow{1} 2 \xrightarrow{1} 3$
 $0 \xrightarrow{1} 1 \xrightarrow{2} 3$
 $0 \xrightarrow{2} 2 \xrightarrow{1} 3$ } for $N=3$
 we have 3 ways



$n=4$

→ for every stair we have 2 cases i.e.



OVERLAP

if $CS == n$
 return 1

if $CS > n$
 return 0

5 ways

$0 \xrightarrow{1} 1 \xrightarrow{1} 2 \xrightarrow{1} 3 \xrightarrow{1} 4$
 $0 \xrightarrow{1} 1 \xrightarrow{1} 2 \xrightarrow{2} 4$
 $0 \xrightarrow{1} 1 \xrightarrow{2} 3 \xrightarrow{1} 4$
 $0 \xrightarrow{2} 2 \xrightarrow{1} 3 \xrightarrow{1} 4$
 $0 \xrightarrow{2} 2 \xrightarrow{2} 4$

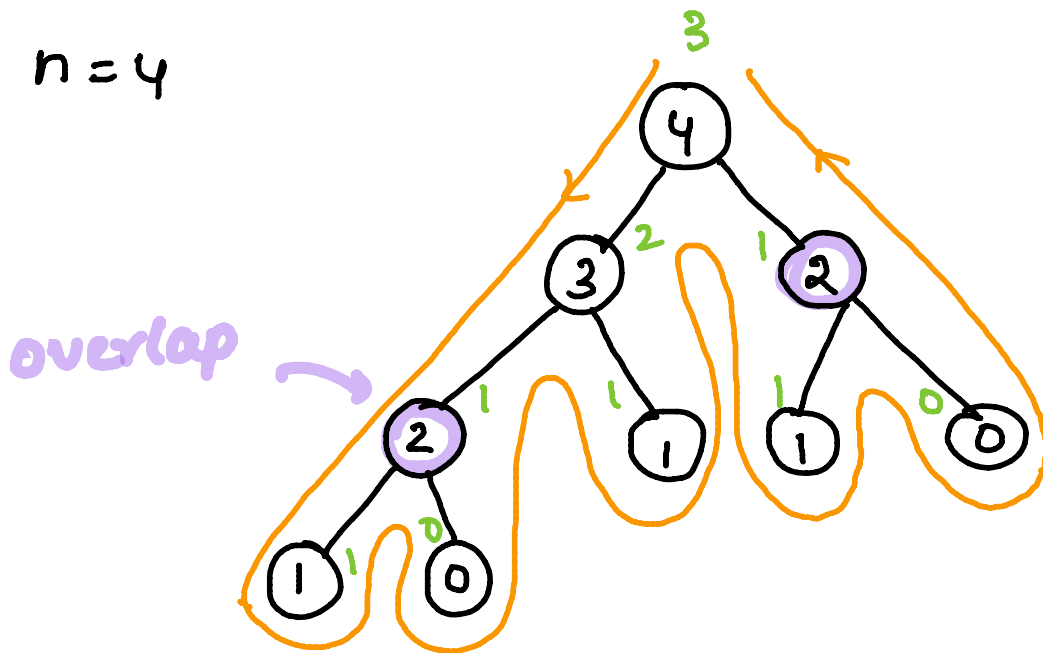
* Here we can see for (2), (3)
 the subproblem is being done multiple times, we can solve using
 dp.

code →

```
1  class Solution {
2  public:
3      int totalWays(int currentStair, int targetStair, unordered_map<int,int> &memo){
4
5          if(currentStair==targetStair){
6              return 1;
7          }
8
9          if(currentStair > targetStair){
10             return 0;
11         }
12
13         int currentKey = currentStair;
14
15         if(memo.find(currentKey)!=memo.end()){
16             return memo[currentKey];
17         }
18
19         int oneStep = totalWays(currentStair+1, targetStair, memo);
20         int twoStep = totalWays(currentStair+2, targetStair, memo);
21
22         memo[currentKey] = oneStep+twoStep;
23
24         return oneStep+twoStep;
25     }
26
27     int climbStairs(int n) {
28         unordered_map<int,int> memo;
29         return totalWays(0,n,memo);
30     }
31 }
32 };
```


② Fibonacci Number → $f(n) = f(n-1) + f(n-2)$ & $f(0) = 0$
 $f(1) = 1$
 $n \geq 0$.

$n = 4$



code →

```

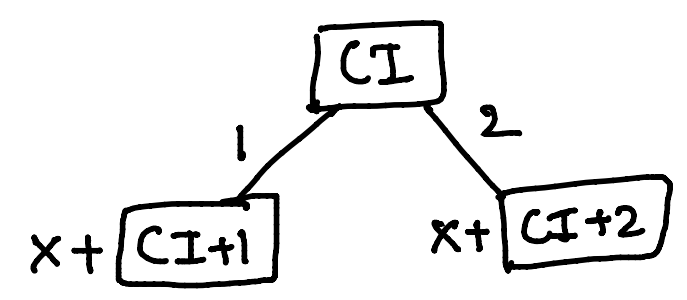
1  class Solution {
2  public:
3      int helper(int n, unordered_map<int,int>&memo){
4
5          if(n<=1){
6              return n;
7          }
8
9          int currentKey = n;
10
11         if(memo.find(currentKey)!=memo.end()){
12             return memo[currentKey];
13         }
14
15
16         int a = helper(n-1,memo);
17         int b = helper(n-2,memo);
18
19         memo[currentKey] = a+b;
20         return memo[currentKey];
21     }
22
23
24     int fib(int n) {
25         unordered_map<int,int>memo;
26         return helper(n,memo);
27     }
28 };

```

③ Min Cost Climbing Stairs →

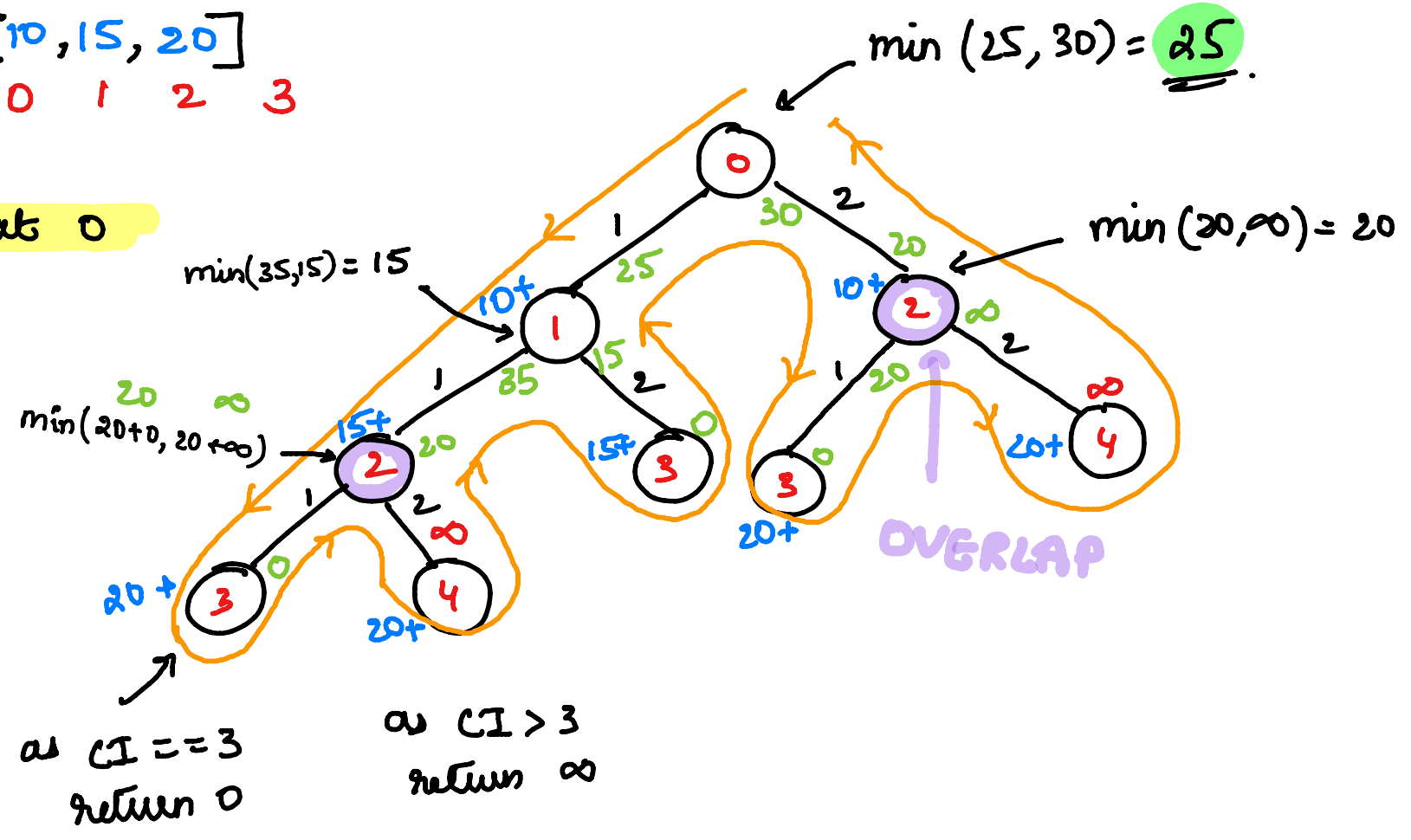
Given costs array, find min cost to reach the end, starting from 0 or 1 & making 1 or 2 jumps.

∴ costs = [_ _ $\overset{\text{CI}}{\underset{\uparrow}{x}}$ _ _]

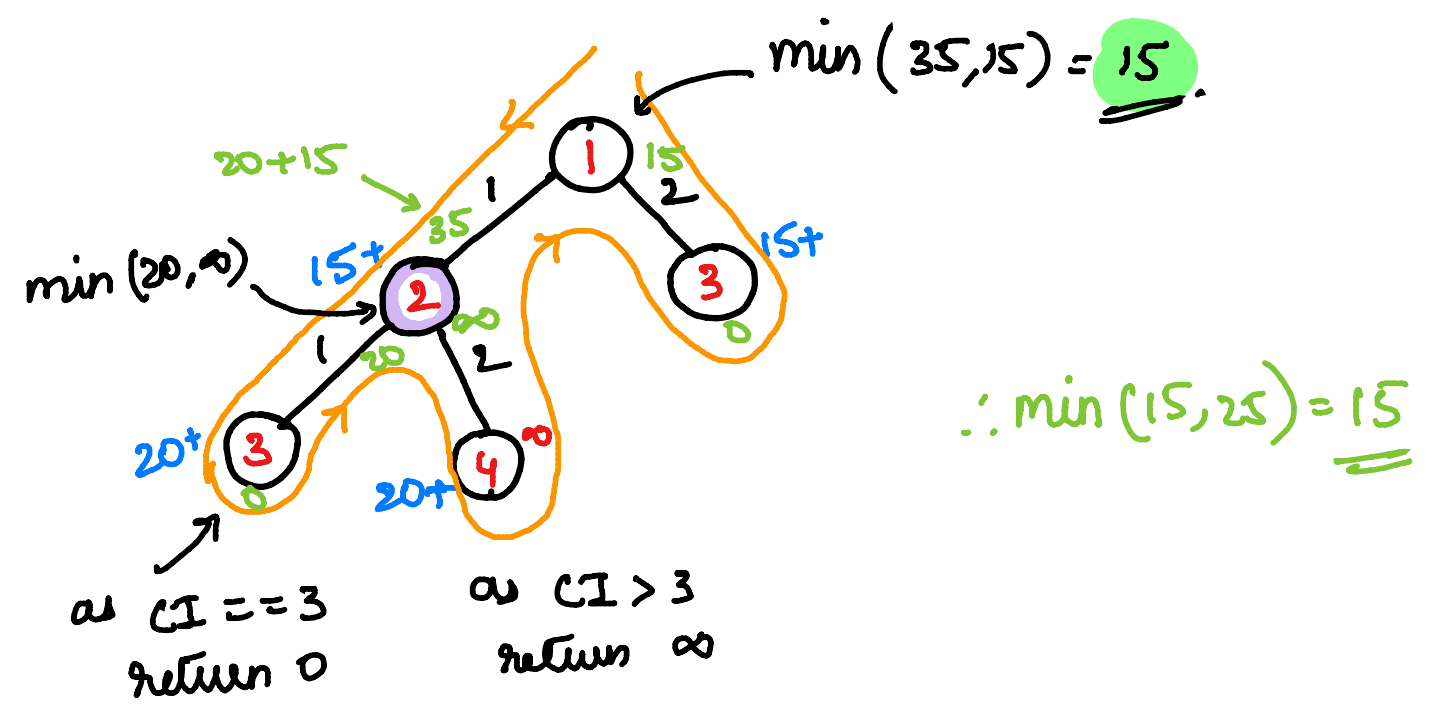


eg cost = [10, 15, 20]
0 1 2 3

starting at 0



starting at 1



code →

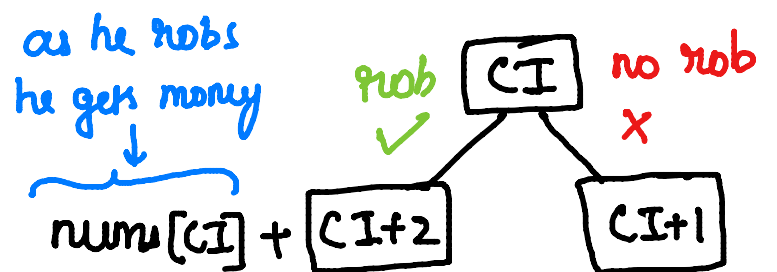
```
1  class Solution {
2  public:
3
4      int minCost(vector<int>&cost, int currentIndex, unordered_map<int,int> &m){
5
6          if(currentIndex == cost.size()){
7              return 0;
8          }
9
10         if(currentIndex > cost.size()){
11             return 1000;    // large values, serves as INFINITY
12         }
13
14         if(m.find(currentIndex)!=m.end()){
15             return m[currentIndex];
16         }
17
18         int oneJump = cost[currentIndex] + minCost(cost,currentIndex+1, m);
19         int twoJump = cost[currentIndex] + minCost(cost,currentIndex+2, m);
20
21         m[currentIndex] = min(oneJump, twoJump);
22         return m[currentIndex];
23     }
24
25     int minCostClimbingStairs(vector<int>& cost) {
26         unordered_map<int,int> m;
27         return min( minCost(cost,0,m), minCost(cost,1,m));
28     }
29 };
```

④ House Robber → Given an array of no. representing money, find max amount, that can be robbed without choosing the adjacent houses.

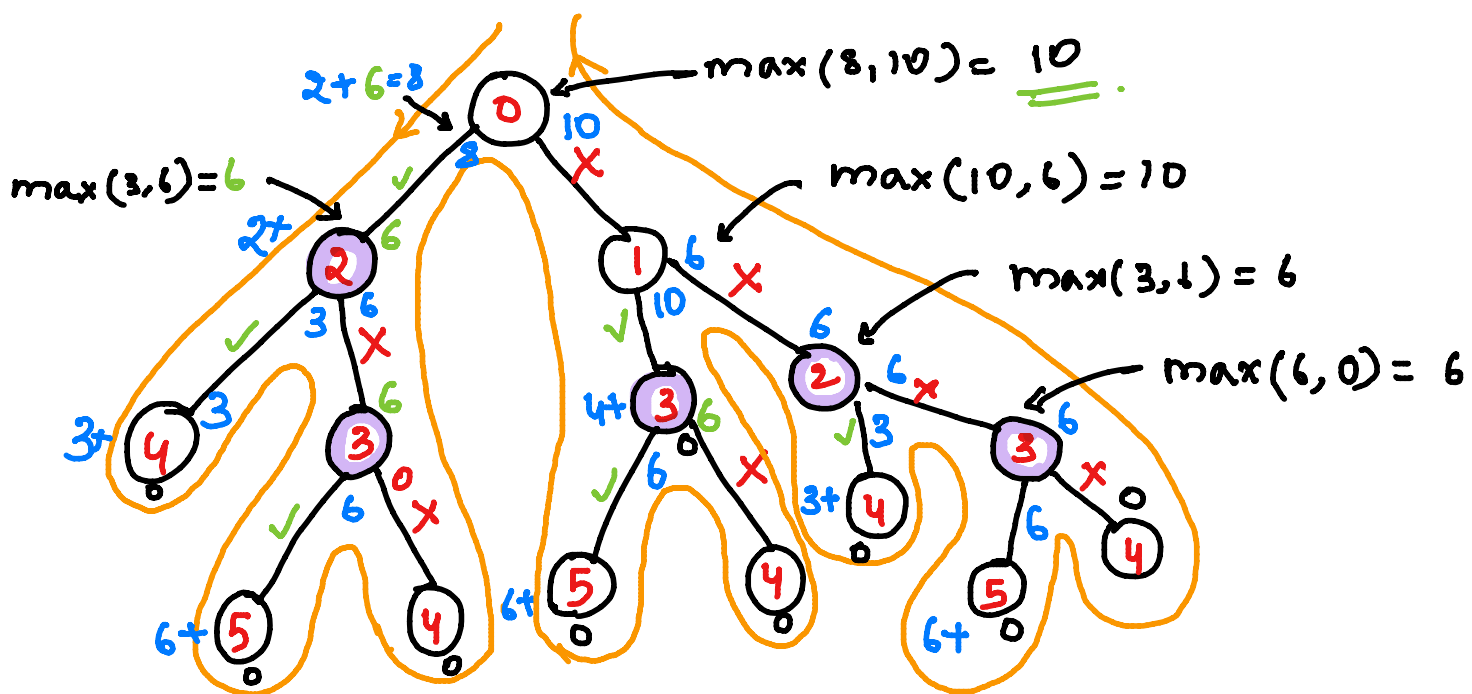
Eg $nums = [2, 7, 9, 3, 1]$
 $\Rightarrow \text{max amount} = 2 + 9 + 1 = 12$
0 → 2 → 4

→ If robber robs house, then $\text{amount} = \text{nums}[CI]$ & $CI = CI + 2$ to avoid adjacent
 else $CI = CI + 1$

ie $nums = [_ _ _ _]$
CI



Eg $[2, 4, 3, 6]$
0 1 2 3



as $CI > 3$
 return 0

∴ at every node find $\text{max}(\text{left}, \text{right})$
 & add it's value to the $\text{nums}[CI]$
 if selected, else continue.

code →

```
1  class Solution {
2  public:
3
4      int helper(vector<int>&nums, int currentIndex, unordered_map<int,int>&m){
5
6          if(currentIndex >= nums.size()){
7              return 0;
8          }
9
10         int currentKey = currentIndex;
11
12         if(m.find(currentKey)!=m.end()){
13             return m[currentKey];
14         }
15
16         int rob = nums[currentKey] + helper(nums, currentIndex+2, m);
17         int noRob = helper(nums, currentIndex+1, m);
18
19         m[currentIndex] = max(rob, noRob);
20
21         return m[currentIndex];
22     }
23
24     int rob(vector<int>& nums) {
25         unordered_map<int,int> m;
26         return helper(nums,0,m);
27     }
28 };

```

⑤ House Robber - II →

In this problem, the approach will be similar to previous one, but the houses are in circle, which means that

* if we start from 1st house, then we can't rob the last house.

* if we start from 2nd house, then we can rob the last house.

* and return max value between 1st house & 2nd house

* if only 1 house is present, then rob it directly.

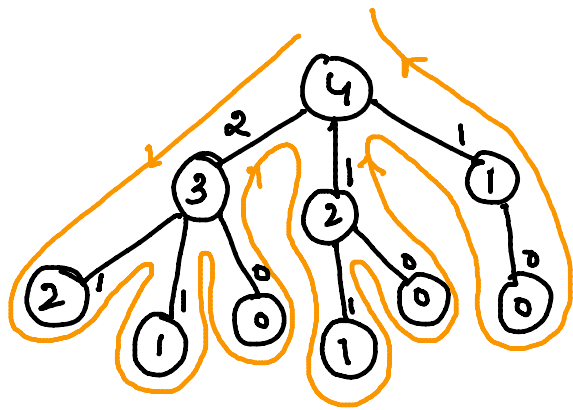
code →

```
1 class Solution {
2     public:
3
4         int helper(vector<int>&nums, int currentIndex, int lastIndex, unordered_map<int,int>&m){
5
6             if(currentIndex > lastIndex){
7                 return 0;
8             }
9
10            int currentKey = currentIndex;
11
12            if(m.find(currentKey)!=m.end()){
13                return m[currentKey];
14            }
15
16            int rob = nums[currentKey] + helper(nums, currentIndex+2, lastIndex, m);
17            int noRob = helper(nums, currentIndex+1, lastIndex, m);
18
19            m[currentIndex] = max(rob, noRob);
20
21            return m[currentIndex];
22        }
23    }
24
25    int rob(vector<int>& nums) {
26
27        int n = nums.size();
28        if(n==1) return nums[0];
29
30        unordered_map<int,int> memo1,memo2;
31        // we can start robbing from 2 houses
32        int firstHouse = helper(nums, 0, n-2, memo1);
33        int secondHouse = helper(nums, 1, n-1, memo2);
34        return max(firstHouse, secondHouse);
35    }
36};
```


⑥ N-th Tribonacci → given n , find T_n

✓ $T_{n+3} = T_n + T_{n+1} + T_{n+2}$ & $n \geq 0$ $T_0 = 0, T_1 = 1, T_2 = 1$.

Eg $n = 4$



4 $(2+1+1)$

code →

```
1 class Solution {
2 public:
3
4     int helper(int n, unordered_map<int,int> &m){
5         if(n<=1){
6             return n;
7         }
8
9         if(n==2){
10            return 1;
11        }
12
13        int currentNum = n;
14
15        if(m.find(currentNum)!=m.end()){
16            return m[currentNum];
17        }
18
19        int a = helper(n-1,m);
20        int b = helper(n-2,m);
21        int c = helper(n-3,m);
22
23        m[currentNum] = a+b+c;
24
25        return m[currentNum];
26    }
27
28    int tribonacci(int n) {
29        unordered_map<int,int>m;
30        return helper(n,m);
31    }
32};
```

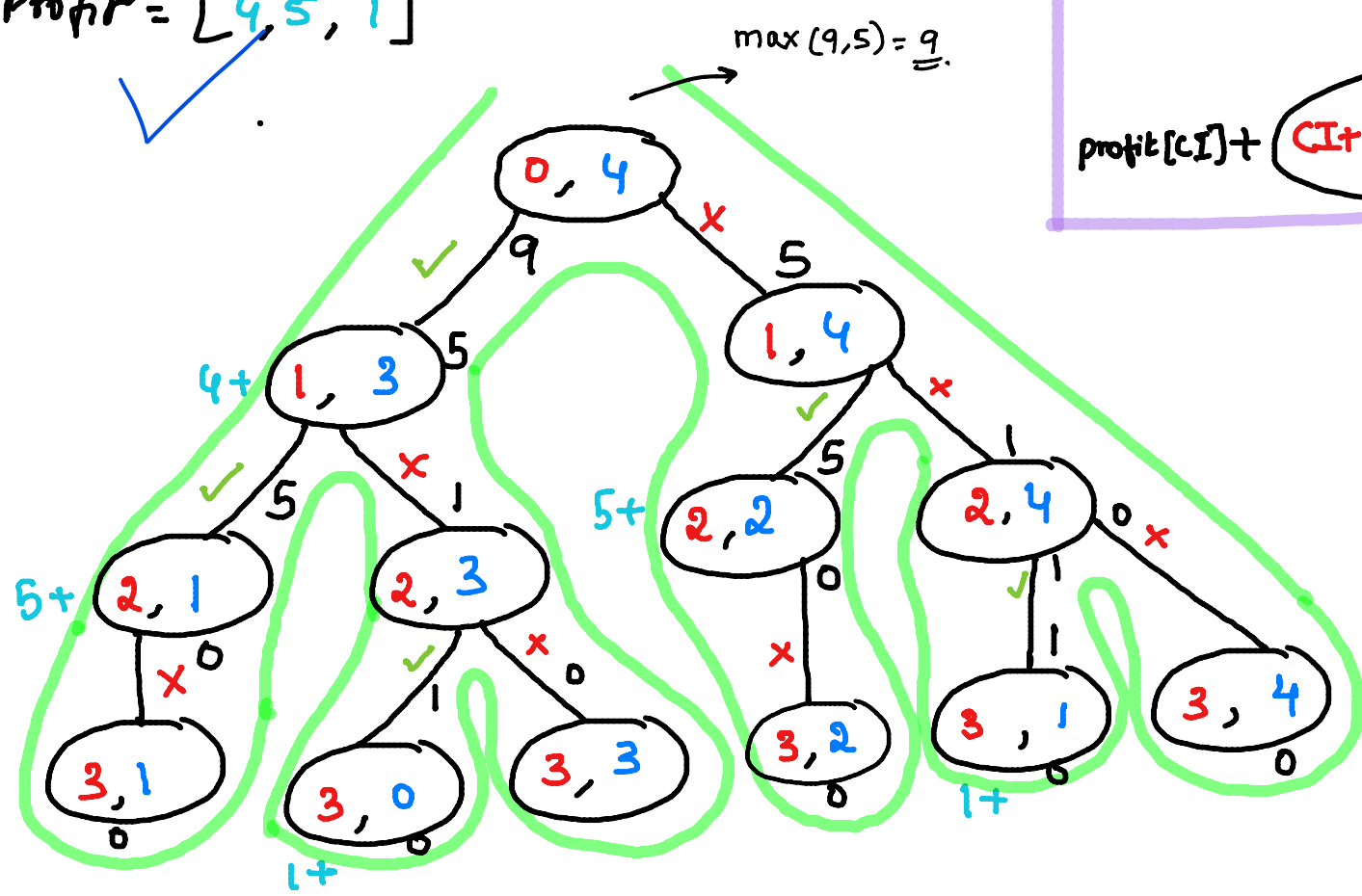
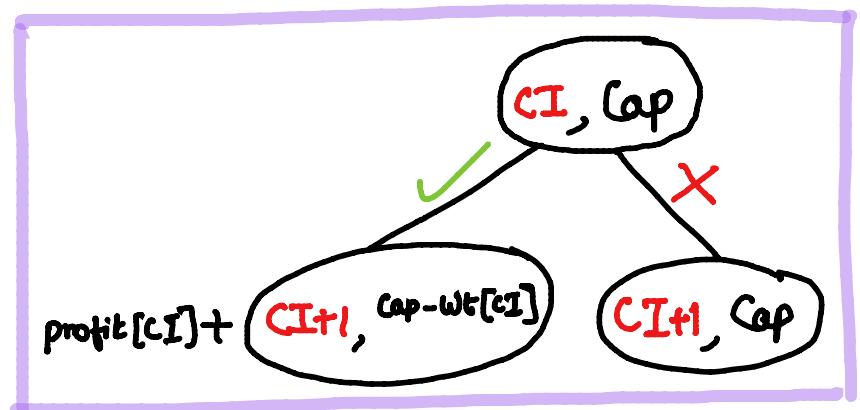
⑦ 0-1 Knapsack Problem → find max profit such that the weight of all items $\leq$ capacity.

Wt = [3, 4, 6, 5]
 profits = [2, 3, 1, 4] → if we select 0 & 3,
 capacity = 8
 then total weight = $wt[0] + wt[3]$
 $= 3 + 5 = 8$.

& profits are $2 + 4 = 6$ That's max profit possible

Eg

Wt = [1, 2, 3]
 Profit = [4, 5, 1]
 capacity = 4



- ∴ at every step,
- if selecting an index then reduce capacity by $wt[CI]$ & add profit $[CI]$ to result
 - if not selecting, increment CI by 1
 - find max(left, right)

code →

```
1  class Solution
2  {
3      public:
4
5      int helper(int W, int wt[], int val[], int n, int curr,
6                  unordered_map<string,int> &memo){
7          if(curr==n) return 0;
8
9          // Instead of Matrix we can use strings as unique keys
10         string currKey = to_string(curr)+"_"+to_string(W);
11
12         if(memo.find(currKey)!=memo.end()) return memo[currKey];
13
14         int currWt = wt[curr];
15         int currVal = val[curr];
16
17         int selected = 0;
18         if(currWt<=W){
19             selected = currVal + helper(W-currWt, wt, val, n, curr+1, memo);
20         }
21
22         int notSelected = helper(W, wt, val, n, curr+1, memo);
23
24         memo[currKey] = max(selected, notSelected);
25         return memo[currKey];
26     }
27
28
29     int knapSack(int W, int wt[], int val[], int n)
30     {
31         unordered_map<string,int> memo;
32         return helper(W, wt, val, n, 0, memo);
33     }
34 };
```

⑧ Partition Equal Subset Sum →

Given an array, find if it can be divided into two subsets whose sum is equal.

Eg $\text{nums} = [1, 5, 11, 5]$ can be divided into $S1 = \{1, 5, 5\}$ & $S2 = \{11\}$
& $\text{sum of } S1 == \text{sum of } S2 \therefore \text{returns True.}$

$\therefore$ initially find sum of elements in array.

1) if sum is odd then return False

2) if sum is even, then proceed.

→ find a subset whose value == $\text{sum}/2$
which means that the other subset will have
value == $\text{sum}/2$.

→ let's ~~say~~ $ts = \text{sum}/2$ (ts is target sum)

→ At every index, we have 2 choices

1) if we select then

$$ts = ts - \text{nums}[CI] \quad \text{current index}$$
$$CI = CI + 1$$

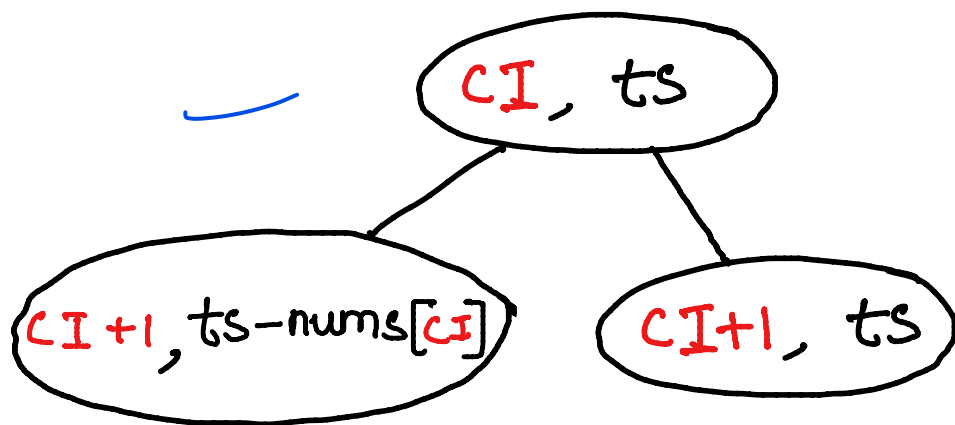
2) if we do not select then $ts = ts$ (i.e. remains same)
 $CI = CI + 1$

3) return OR of left & right branch.

$\Rightarrow \text{nums} = [1, 5, 11, 5]$

$\text{sum} = 22$

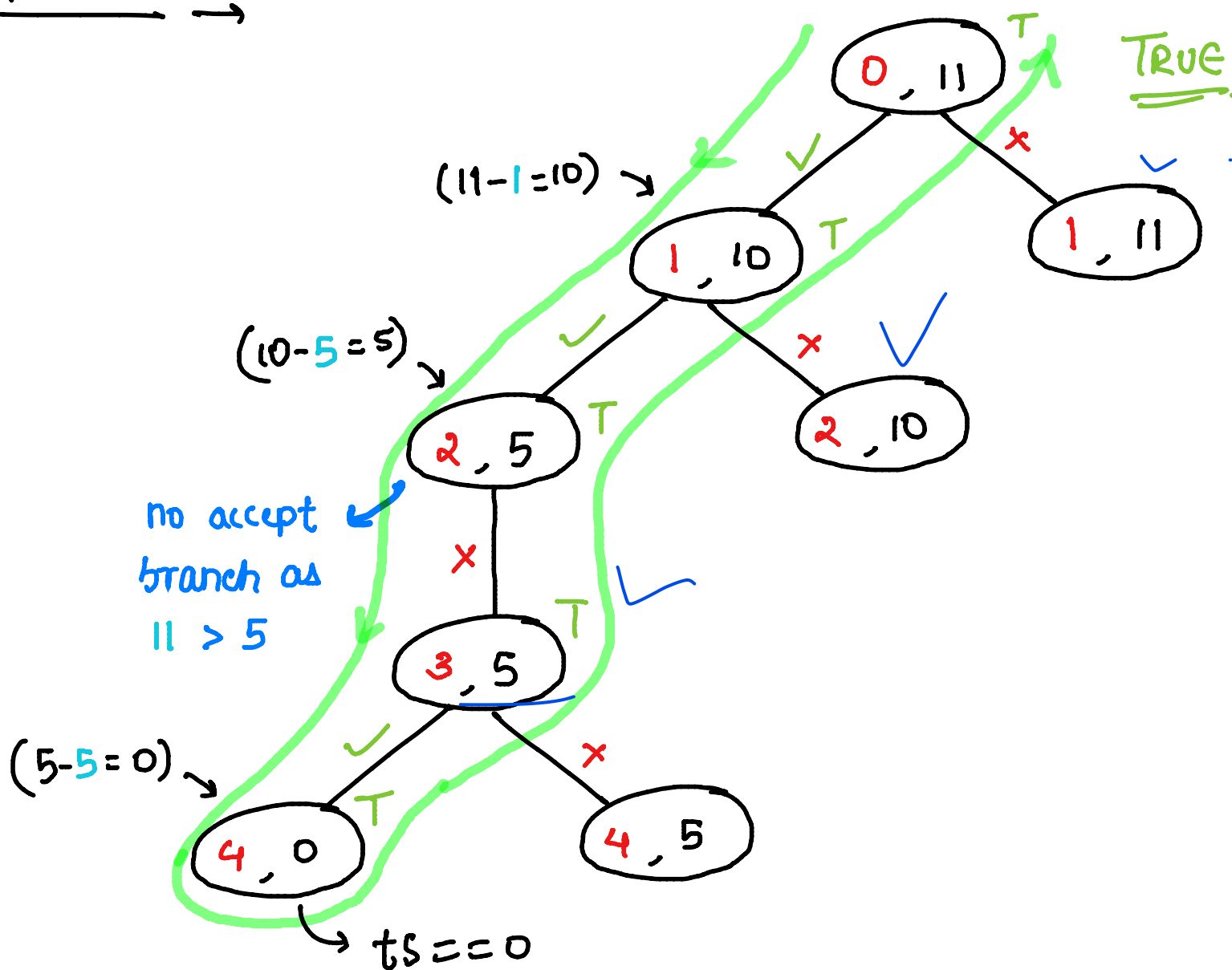
$\text{ts} = 11$ ✓



Here $\text{sum} \% 2 == 0$

$\therefore$ dividing into 2 subsets is possible.

Explanation →



$\Rightarrow$ that subset is found
 & return True

as we are using OR, one True branch is sufficient

code →

```
1  class Solution {
2  public:
3
4      bool isPossible(int targetSum,int currentIndex, vector<int>&nums,
5                      unordered_map<string, bool> &memo){
6
7          if(targetSum == 0)
8              return true;
9
10         if(currentIndex >= nums.size())
11             return false;
12
13         string currentKey = to_string(currentIndex)+"_"+to_string(targetSum);
14
15         if(memo.find(currentKey)!=memo.end()){
16             return memo[currentKey];
17         }
18
19         bool possible = false;
20
21         if(nums[currentIndex]<=targetSum)
22             possible = isPossible(targetSum-nums[currentIndex], currentIndex+1, nums, memo);
23
24         // if already Possible then return True directly
25         if(possible){
26             memo[currentKey] = possible;
27             return true;
28         }
29
30         bool notPossible = isPossible(targetSum, currentIndex+1, nums, memo);
31
32         memo[currentKey] = possible||notPossible;
33         return memo[currentKey];
34     }
35
36     bool canPartition(vector<int>& nums) {
37
38         int total = 0;
39         for(auto it:nums) total+= it;
40
41         if(total%2!=0) return false;
42
43         unordered_map<string, bool> memo;
44         return isPossible(total/2,0, nums,memo);
45     }
46 };
```

9) Target Sum →

Given an array & target, find the number of ways to reach target by using + or - before each element in array.

Eg

Input: nums = [1,1,1,1,1], target = 3

Output: 5

Explanation: There are 5 ways to assign symbols to make the sum of nums be target 3.

-1 + 1 + 1 + 1 + 1 = 3

+1 - 1 + 1 + 1 + 1 = 3

+1 + 1 - 1 + 1 + 1 = 3

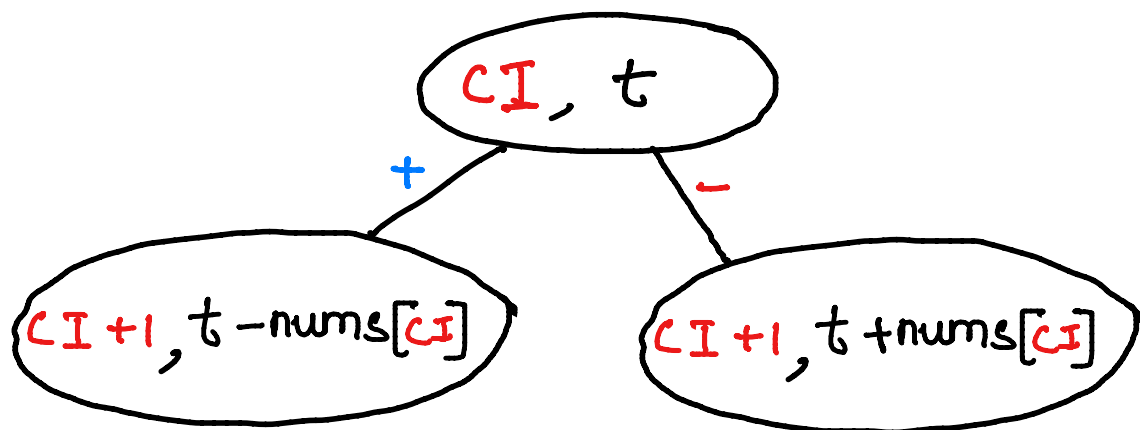
+1 + 1 + 1 - 1 + 1 = 3

+1 + 1 + 1 + 1 - 1 = 3

→ at every index we can use + or - sign

if + then $t = t - (+ \text{nums}[CI]) \Rightarrow t - \text{nums}[CI]$

if - then $t = t - (-\text{nums}[CI]) \Rightarrow t + \text{nums}[CI]$



→ at every node, return the sum of values from left & right. Because we need to find the total number of ways.

code →

```
1 class Solution {
2 public:
3     int totalWays(int currentIndex, vector<int>&nums, int target, unordered_map<string,int> &memo){
4
5         if(target==0 and currentIndex==nums.size()){
6             return 1;
7         }
8
9         if(currentIndex>=nums.size() and target!=0){
10             return 0;
11         }
12
13         string key = to_string(currentIndex)+"_"+to_string(target);
14
15         if(memo.find(key)!=memo.end()){
16             return memo[key];
17         }
18
19         int plus = totalWays(currentIndex+1, nums, target-nums[currentIndex],memo);
20
21         int minus = totalWays(currentIndex+1, nums, target+nums[currentIndex],memo);
22
23         memo[key] = plus+minus;
24
25         return plus+minus;
26     }
27
28     int findTargetSumWays(vector<int>& nums, int target) {
29         unordered_map<string,int> memo;
30         return totalWays(0,nums,target,memo);
31     }
32 };
```


⑩ Count number of subsets with given difference →

→ This is similar to Target Sum.

given the difference between two subsets, and an array
find no. of subsets with the difference.

Approach →

Let's say $s_1 - s_2 = \text{difference (given)}$ — (1)

we can calculate sum of every element, say sum

& it can be said that for 2 subsets s_1 & s_2

$s_1 + s_2 = \text{sum}$. — (2)

Now (1) + (2) $\Rightarrow 2(s_1) = \text{difference} + \text{sum}$

$s_1 = (\text{difference} + \text{sum}) / 2$.

→ Implement Target Sum with target value = s_1 .

⑪ Delete and Earn →

You are given an integer array `nums`. You want to maximize the number of points you get by performing the following operation any number of times:

- Pick any `nums[i]` and delete it to earn `nums[i]` points. Afterwards, you must delete **every** element equal to `nums[i] - 1` and **every** element equal to `nums[i] + 1`.

Return the **maximum number of points** you can earn by applying the above operation some number of times.

Eg `nums = [2, 2, 3, 3, 3, 4]`

→ if we start deleting **2**, then $\text{result} = 2 + 2 = 4$

then `nums = [3, 3, 3, 4]`

& we need to delete all $2+1$ & $2-1 \Rightarrow \text{nums} = [4]$

→ if we delete 4, then $\text{result} = 4 + 4 = \underline{8}$.

& `nums = []`

(or)

→ if we start deleting **3**, then $\text{result} = 3 + 3 + 3 = 9$.

then `nums = [2, 2, 4]`

& we need to delete all $3-1$ & $3+1 \Rightarrow \text{nums} = []$

$\therefore \text{result} = \underline{9}$.

(or)

→ if we start deleting **4**, then $\text{result} = 4$

then `nums = [2, 2, 3, 3, 3]`

& we need to delete all $4+1$ & $4-1 \Rightarrow \text{nums} = [2, 2]$

→ if we delete 2, then $\text{result} = 4 + 4 = \underline{8}$.

& `nums = []`

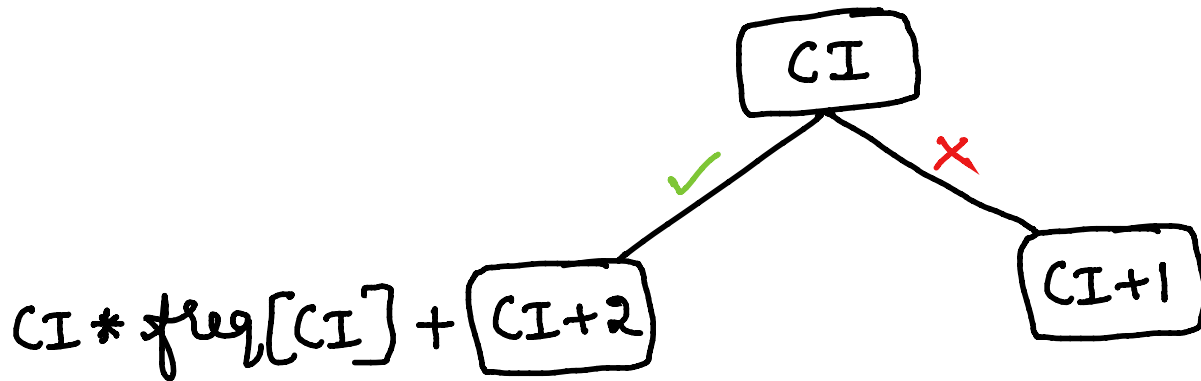
→ we can store frequency of each element and use the similar approach

nums = [2, 2, 3, 3, 3, 4]

freq =

0	0	2	3	1
---	---	---	---	---

 0 1 2 3 4
 ↑
 CI



code →

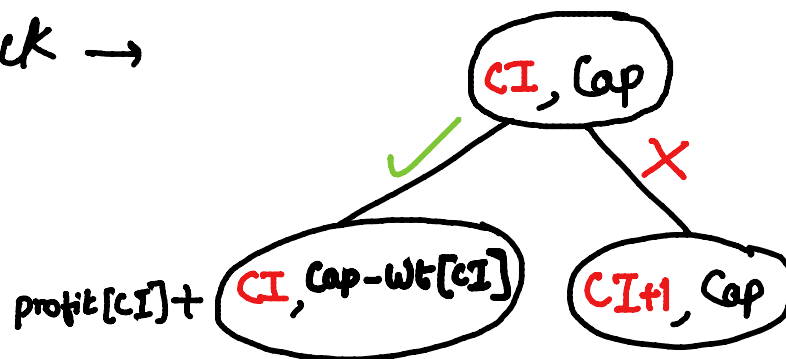
```
1 class Solution {
2 public:
3
4     int maxPoints(vector<int>& freq, int currentIndex, unordered_map<int,int>&memo){
5
6         if(currentIndex >= freq.size()) return 0;
7
8         int key = currentIndex;
9
10        if(memo.find(key) != memo.end()) return memo[key];
11
12        int Delete = currentIndex*freq[currentIndex] + maxPoints(freq, currentIndex+2, memo);
13        int NotDelete = maxPoints(freq, currentIndex+1, memo);
14
15        memo[key] = max(Delete, NotDelete);
16
17        return memo[key];
18    }
19
20    int deleteAndEarn(vector<int>& nums) {
21
22        int maxi = *max_element(nums.begin(), nums.end());
23        vector<int> freq(maxi+1, 0);
24
25        for(auto i: nums) freq[i]++;
26
27        unordered_map<int,int> memo;
28
29        return maxPoints(freq, 0, memo);
30    }
31 };
```

⑫ Unbounded Knapsack → Similar to 0-1 Knapsack but allows us to choose an item more than once

Eg
wt = [2, 1]
values = [1, 1]
capacity = 3

if bounded knapsack then profit = 2. (2, 1)
if unbounded knapsack then profit = 3. (1, 1, 1)

for unbounded Knapsack →



Code →

```
1 class Solution{
2 public:
3     int helper(int W, int wt[], int val[], int N, int curr, vector<vector<int>>&memo){
4
5         if(W==0) return 0;
6         if(curr==N) return 0;
7
8         if(memo[curr][W]!=-1) return memo[curr][W];
9
10        int currWt = wt[curr];
11        int currVal = val[curr];
12
13        int selected = 0;
14        if(currWt<=W){
15            selected = currVal + helper(W-currWt, wt, val, N, curr, memo);
16        }
17
18        int notSelected = helper(W, wt, val, N, curr+1, memo);
19
20        memo[curr][W] = max(selected, notSelected);
21        return memo[curr][W];
22    }
23
24    int knapSack(int N, int W, int val[], int wt[])
25    {
26        vector<vector<int>> memo( N , vector<int> (W+1, -1));
27        return helper(W, wt, val, N, 0, memo);
28    }
29 };
```

⑬ Coin Change II → (Similar to unbounded knapsack)

Given an array of coins & amount, find total number of ways / combinations to make up that amount.

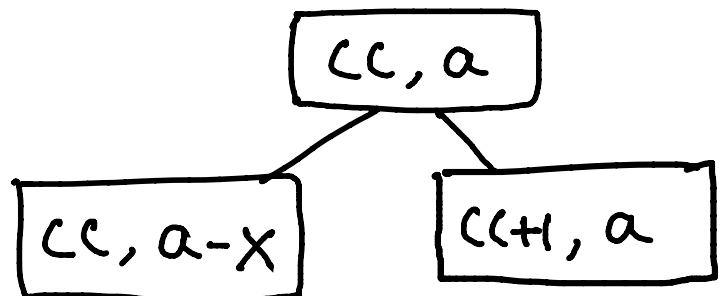
Eg coins = [1, 2, 5]
amount = 5

4 ways

$\left\{ \begin{array}{l} 1+1+1+1+1 \\ 1+1+1+2 \\ 1+2+2 \\ 5 \end{array} \right.$

coins = [_ $\overset{x}{\text{---}}$ _ _ _] $x = \text{coins}[cc]$

↑
current coin cc, a amount



code →

```
1 class Solution {
2 public:
3
4     int totalWays(int currentIndex, vector<int>& coins, int amount, vector<vector<int>>& memo){
5         if(amount == 0) return 1; // amount==0 means that target is reached so return 1
6         if(currentIndex >= coins.size()) return 0; //if index is out of bounds then return 0
7
8         if(memo[currentIndex][amount] != -1) return memo[currentIndex][amount];
9
10        int consider = 0;
11        if(coins[currentIndex] <= amount){
12            consider = totalWays(currentIndex, coins, amount - coins[currentIndex], memo);
13        }
14        int notConsider = totalWays(currentIndex+1, coins, amount, memo);
15
16        memo[currentIndex][amount] = consider + notConsider;
17        return memo[currentIndex][amount];
18    }
19
20    int change(int amount, vector<int>& coins) {
21        vector<vector<int>> memo(coins.size()+1, vector<int>(amount+1, -1));
22        return totalWays(0, coins, amount, memo);
23    }
24 };
```


14) Coin Change →

(Similar to unbounded knapsack)

Given an array of coins & amount, find fewest number of coins to make up that amount, return -1 if it's not possible

Eg coins = [1, 2, 5]
amount = 11

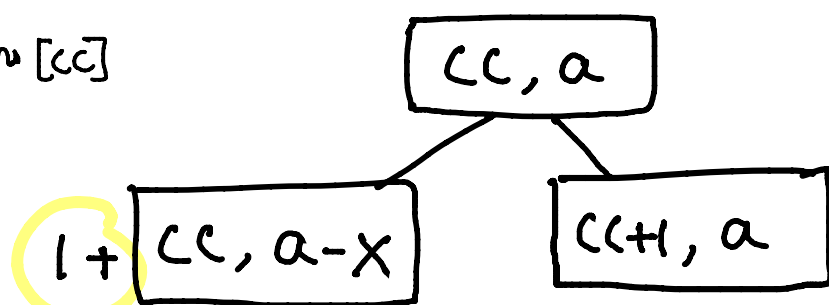
for 11 ⇒

$\underbrace{1 + \dots + 1}_{11 \text{ times}} = 11$
 $1 + 2 + 2 + 2 + 2 + 2 = 11$
 $1 + 5 + 5 = 11$

out of all the ways last has min coins.

coins = [- $\overset{x}{\curvearrowright}$ - - -] $x = \text{coins}[cc]$

current coin $\uparrow$ cc, a amount



↳ this contributes to counting coins.

code →

```
1 class Solution {
2 public:
3
4     int minimumCoins(int currentIndex, vector<int>& coins, int amount, vector<vector<int>>& memo){
5         if(amount == 0) return 0;
6         if(currentIndex >= coins.size()) return 100000; //Any Max Value outside boundary
7
8         if(memo[currentIndex][amount] != -1) return memo[currentIndex][amount];
9
10        int consider = 100000;
11        if(coins[currentIndex] <= amount){
12            consider = 1 + minimumCoins(currentIndex, coins, amount - coins[currentIndex], memo);
13        }
14
15        int notConsider = minimumCoins(currentIndex + 1, coins, amount, memo);
16
17        memo[currentIndex][amount] = min(consider, notConsider);
18        return memo[currentIndex][amount];
19    }
20
21    int coinChange(vector<int>& coins, int amount) {
22
23        vector<vector<int>> memo(coins.size() + 1, vector<int>(amount + 1, -1));
24        int ans = minimumCoins(0, coins, amount, memo);
25
26        return (ans == 100000) ? -1 : ans;
27    }
28 }
```

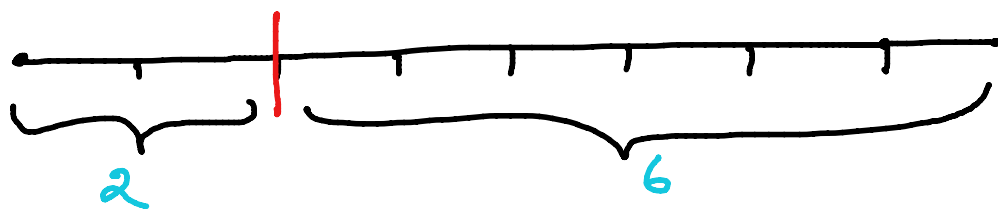

⑮ Rod Cutting →

Given a rod of length N and array of prices. find the max value that can be obtained by cutting rod.

Eg $N=8$ prices = $[1, 5, 8, 9, 10, 17, 17, 20]$

1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7

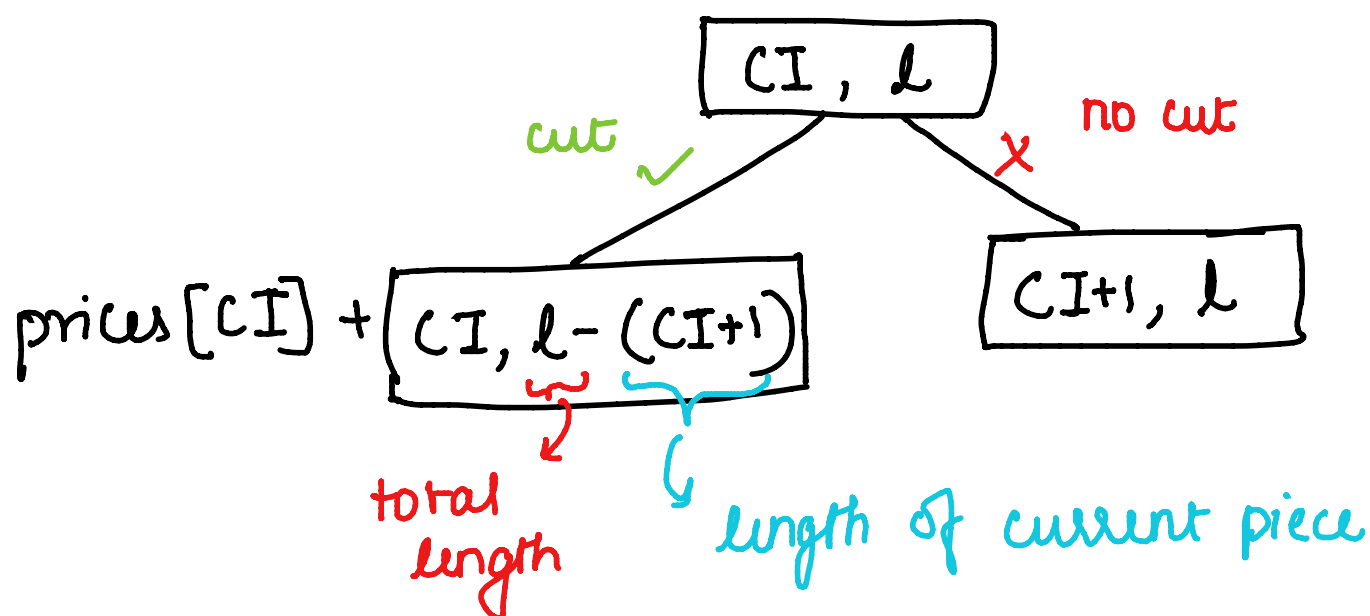
* price of a piece is $prices[CI]$, whose length is $CI+1$



if we cut our rod into 2 pieces of length 2, 6 we get max value of $5+17$ is 22.

→ there might be other ways, but this particular configuration returns max value.

* At any instance length of current piece is $CI+1$



code →

```
1
2 class Solution{
3     public:
4         int maxProfit(int price[],int currentIndex, int n, vector<vector<int>>&memo){
5             if(n==0) return 0;
6             if(currentIndex>=n) return 0;
7
8             if(memo[currentIndex][n]!=-1) return memo[currentIndex][n];
9
10            int selected = 0;
11            if(currentIndex+1<=n){
12                selected = price[currentIndex]+maxProfit(price, currentIndex, n-(currentIndex+1), memo);
13            }
14            int notSelected = maxProfit(price, currentIndex+1, n, memo);
15
16            memo[currentIndex][n] = max(selected, notSelected);
17            return memo[currentIndex][n];
18        }
19        int cutRod(int price[], int n) {
20            vector<vector<int>> memo(n+1, vector<int>(n+1,-1));
21            return maxProfit(price,0,n,memo);
22        }
23    };
```

